2D Object Detection with COCO-Pretrained Detectors: Generalization from COCO to KITTI (Track A)

Adam Sabry · CMPE-249 Sec 33, Fall 2026
Code, results and command log: https://github.com/AdamSabry1233/2D-Object-Detection

1. Objective and approach

I ask how well three COCO-pretrained detectors from different architecture families transfer to KITTI driving scenes *without retraining*, and what each costs in speed and memory. I evaluate YOLO11s, RT-DETR-L and Faster R-CNN R50-FPN on COCO val2017 and the KITTI val split, restricted to the three classes both datasets share (person, car, truck), and score every run with pycocotools. The two analyses are cross-dataset generalization (Section 5) and the speed-accuracy trade-off with a short architecture discussion (Section 6); Section 7 is a failure analysis. Every number below is computed from files in the repository; `docs/claims_ledger.md` lists each claim with its source file and a status (measured, derived, hypothesis or unverified), and I label statements as measured or hypothesis in the text. AP values are fractions with three decimals.

2. Models and datasets

Table 1 lists the models, Table 2 the datasets (COCO [1]; KITTI [2], [3], [15]) and Table 3 the KITTI-to-COCO label mapping used to compare them.

Table 1. The three evaluated detectors. Parameter counts are from `results/params/model_params.json`. YOLO11 has no paper; its documentation is the citation [10]. Architecture references: [4]–[9].

Model	Family / design	Checkpoint	Params (M)	NMS	Input handling
YOLO11s	single-stage CNN, anchor-free	yolo11s.pt (Ultralytics)	9.46	yes (IoU 0.7)	imgsz 640, aspect-preserving letterbox (KITTI: 224×640)
RT-DETR-L	transformer: hybrid encoder + query decoder	rtdetr-l.pt (Ultralytics)	32.97	none	imgsz 640, stretched to 640×640, no padding
Faster R-CNN R50-FPN	two-stage, anchor-based (RPN + RoI head)	torchvision COCO_V1	41.76	yes (IoU 0.5)	internal resize, min side 800 / max side 1333 (KITTI: 402×1333; tensor 416×1344)

Table 2. Datasets. COCO is scored on the full val2017 split; KITTI on the 1,496-image val split of the Ultralytics repackaging [15], converted to COCO format by `scripts/convert_kitti_to_coco.py`.

Dataset	Split	Images	Objects scored (person / car / truck)	Notes
COCO	val2017	5000	13109 total (non-crowd)	80 classes; 3 shared classes used for comparison
KITTI	val (Ultralytics repackaging)	1496	921 / 6271 / 223 (+388 ignore regions)	converted to COCO format, COCO category ids

Table 3. KITTI-to-COCO label mapping. The cross-dataset comparison uses person (COCO id 1), car (3) and truck (8).

KITTI class	Mapped to	Reason
car	car (3)	direct match
van	car (3)	COCO has no van class
truck	truck (8)	direct match
pedestrian	person (1)	direct match
person_sitting	person (1), ignore region (iscrowd=1)	not scored: neither a hit nor a false positive
cyclist	person (1), ignore region (iscrowd=1)	box covers rider plus bike, but COCO models predict person and bicycle separately; scoring it would create artificial false positives
tram, misc	dropped	no matching class among the three scored

3. Experimental setup, hardware and reproducibility

Hardware and software. NVIDIA GeForce RTX 4050 Laptop GPU (6 GB VRAM, 45 W power cap, driver 610.60), 15.3 GB RAM, Windows 11 (Git Bash). Python 3.12.10, torch 2.6.0+cu124, torchvision 0.21.0+cu124, ultralytics

8.4.198, pycocotools 2.0.11, numpy 2.5.4. These come from the `hardware` and `versions` fields stored in every result file.

Protocol. YOLO11s and RT-DETR-L run through the Ultralytics package [12], Faster R-CNN through torchvision [13], and all predictions are scored with pycocotools [14]. Confidence threshold 0.001 and at most 100 detections per image for all models, so that AP is computed over the full precision-recall curve. NMS IoU is 0.7 for YOLO11s, 0.5 for Faster R-CNN, and none for RT-DETR-L. **Input handling differs by model.** YOLO11s (imgsz 640) is letterboxed with minimal padding, so a KITTI image enters as 224×640. RT-DETR-L (also imgsz 640) is *not* letterboxed: the installed Ultralytics 8.4.198 sets `scale_fill = True` in its RT-DETR predictor, so every image is stretched to 640×640 without padding; I confirmed both shapes by running each model's own preprocessing on a KITTI image. Faster R-CNN uses torchvision's internal resize (min side 800, max side 1333): 402×1333 for KITTI, in a tensor padded to 416×1344. The three models therefore see KITTI at three different effective inputs (Section 5.5). Ultralytics class ids 0–79 are mapped to COCO ids with `COCO80_TO_91`. **Latency** is the mean time per image of preprocessing + model + NMS + copy to CPU, measured after 10 warmup images with CUDA synchronization; image decoding is not timed. **Peak memory** is `torch.cuda.max_memory_allocated` (tensor allocations only, excluding the CUDA context). Because the GPU is a power-limited laptop part, I treat latency differences under about 20% as noise.

Commands (the `command` field of each result file; the conversion command is from `COMMANDS.md`):

```
python scripts/convert_kitti_to_coco.py
python scripts/eval_detector.py --model yolo11s --dataset coco
python scripts/eval_detector.py --model yolo11s --dataset kitti
python scripts/eval_detector.py --model rtdetr-l --dataset coco
python scripts/eval_detector.py --model rtdetr-l --dataset kitti
python scripts/eval_detector.py --model frcnn --dataset coco
python scripts/eval_detector.py --model frcnn --dataset kitti
python scripts/eval_detector.py --model yolo11s --dataset kitti --imgsz 1280
```

Reproduction and evidence. Each `results/<model>_<dataset>.json` stores the metrics, latency, memory, configuration, versions, hardware and the exact command that produced it; `COMMANDS.md` lists all commands in order, and the code is in `scripts/` of the repository above. Raw detections (large) and datasets are kept outside the repository. A 50-image smoke test is excluded from all results. No training was done, so the KITTI val split carries no leakage concern.

4. Quantitative results

Sanity check. Table 4 compares my 80-class COCO AP with the published values. All three agree within 0.015 AP, which I take as evidence that the label mapping and scoring pipeline are correct. YOLO11s has the largest gap (-0.015); a possible cause is that my per-image `predict` settings differ from the vendor's validation settings (hypothesis, not tested). The published YOLO11s page does not name the split [10].

Table 4. 80-class COCO AP@[.50:.95] on val2017, measured vs. published.

Model	Measured AP (80 classes)	Published AP	Difference	Published source
YOLO11s	0.455	0.470	-0.015	[10]
RT-DETR-L	0.526	0.530	-0.004	[11]
Faster R-CNN	0.372	0.370	+0.002	[13] (weights metadata)

Main results. Table 5 gives the 3-class results and Table 6 the cost. Figure 1 shows the headline result: every detector loses a large share of its AP on KITTI.

Table 5. 3-class (person, car, truck) results. AP = AP@[.50:.95]; s/m/l = small/medium/large objects; AR100 = recall with up to 100 detections. The last row is the separate resolution experiment (Section 5.5); the model is the same YOLO11s checkpoint. Truck on KITTI has only 223 instances, so its AP is noisy.

Model	Data	AP	AP50	AP75	APs	APm	APl	AR100	AP person	AP car	AP truck
YOLO11s	COCO	0.463	0.657	0.499	0.277	0.556	0.697	0.606	0.586	0.448	0.355
YOLO11s	KITTI	0.241	0.475	0.220	0.137	0.218	0.358	0.419	0.196	0.438	0.090
RT-DETR-L	COCO	0.524	0.744	0.561	0.339	0.611	0.753	0.694	0.615	0.499	0.457
RT-DETR-L	KITTI	0.285	0.531	0.269	0.222	0.272	0.404	0.535	0.201	0.481	0.173
Faster R-CNN	COCO	0.418	0.688	0.438	0.286	0.490	0.562	0.584	0.518	0.411	0.325
Faster R-CNN	KITTI	0.266	0.520	0.245	0.280	0.247	0.369	0.505	0.207	0.423	0.169
YOLO11s, imgsz 1280 (resolution exp.)	KITTI	0.262	0.497	0.245	0.215	0.245	0.355	0.457	0.197	0.458	0.131

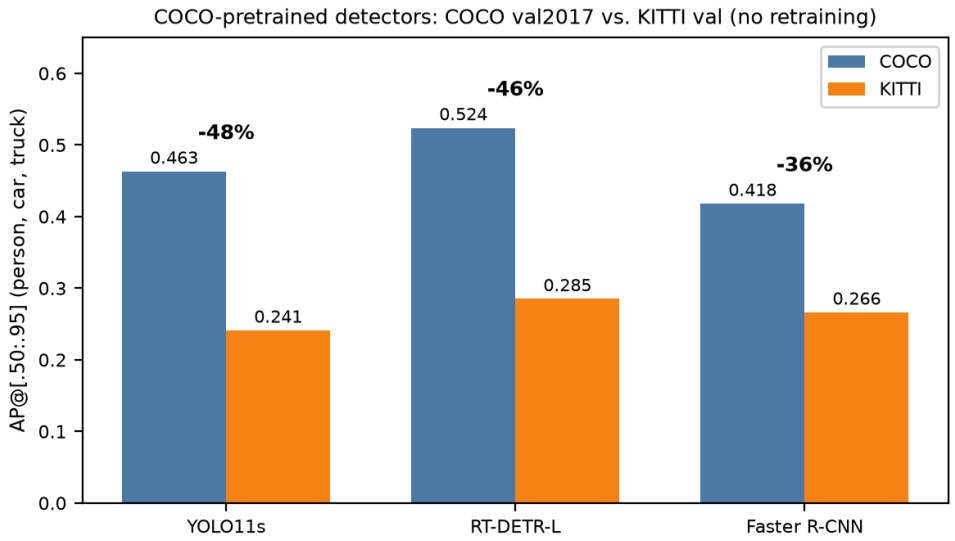


Figure 1. 3-class AP@[.50:.95] on COCO val2017 and KITTI val for the three COCO-pretrained detectors (no retraining). Percentages are the relative change COCO → KITTI. Conf 0.001, max 100 detections, NMS IoU as in Table 1. Caveat: models see different input scales and no confidence intervals were computed, so AP gaps of about 0.02 between models are not established.

Table 6. Latency, throughput and peak GPU memory (mean over images after 10 warmup images; 45 W laptop GPU, differences under about 20% treated as noise).

Model	Data	Input	Mean ms	Median ms	p95 ms	FPS	Peak GPU mem (GB)
YOLO11s	COCO	imgsz=640	15.79	14.24	24.4	63.3	0.126
YOLO11s	KITTI	imgsz=640	18.8	15.59	34.17	53.2	0.087
RT-DETR-L	COCO	imgsz=640	34.77	32.27	46.53	28.8	0.245
RT-DETR-L	KITTI	imgsz=640	33.95	32.08	43.13	29.5	0.245
Faster R-CNN	COCO	internal 800/1333	93.94	94.2	105.08	10.6	0.696
Faster R-CNN	KITTI	internal 800/1333	63.09	62.61	67.38	15.9	0.452

5. Analysis 1: cross-dataset generalization

5.1 Overall. *Expected:* a drop on a different domain, with the COCO ranking roughly preserved. *Observed (derived):* 3-class AP falls by 47.9% for YOLO11s, 45.6% for RT-DETR-L and 36.3% for Faster R-CNN (Figure 1), and the ranking changes: RT-DETR-L > YOLO11s > Faster R-CNN on COCO, but RT-DETR-L > Faster R-CNN > YOLO11s on KITTI. RT-DETR-L and Faster R-CNN differ by 0.019 AP on KITTI (0.285 vs 0.266); without confidence intervals I treat this as too close to call. The AP75/AP50 ratio also falls for all models on KITTI (0.76 → 0.46, 0.75 → 0.51, 0.64 → 0.47 for YOLO11s, RT-DETR-L, Faster R-CNN), so objects that pass IoU 0.5 are less often tight enough to pass 0.75 (measured); why is untested.

5.2 By class. Figure 2 shows that the drop is concentrated in two classes. Person AP changes by -66.7%, -67.3% and -60.0% and truck AP by -74.5%, -62.2% and -48.2% (YOLO11s, RT-DETR-L, Faster R-CNN), while car AP

changes by only -2.4%, -3.7% and +2.9%. The truck numbers rest on 223 objects and are noisy; I did not test whether label-definition differences (for example vans merged into car) contribute.

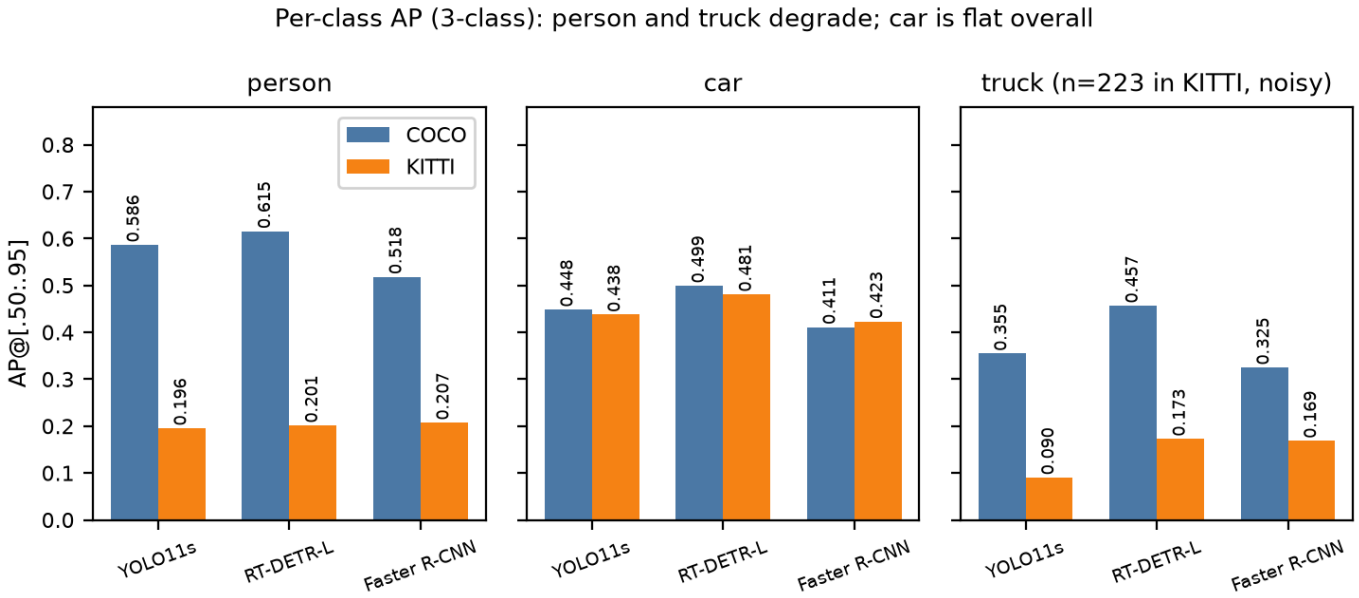


Figure 2. Per-class AP@[.50:.95] (3-class) on COCO and KITTI. Person and truck degrade; car is flat overall (but see Table 7: at matched sizes car AP is lower on KITTI). Truck n=223 on KITTI, so its AP is noisy; KITTI person n=921. Same detection thresholds as Figure 1.

5.3 By size and class × size. Table 7 separates class from size. At matched sizes car AP is *lower* on KITTI in 9 of 9 reliable cells, even though pooled car AP looks flat; the closest to flat is Faster R-CNN on large cars (0.564 vs 0.551, -2.4%). Person AP drops at every size, by 50% to 93% across models and sizes, and small KITTI persons score only 0.024–0.039 AP for all three models, so pedestrians fail at every size (a class-level problem, not only a size problem; measured).

Table 7. AP by class and object size, COCO → KITTI, with object counts. Size buckets are COCO's (small < 32^2 px, medium < 96^2 px) but COCO area comes from segmentation masks and the converted KITTI file uses box area. AP is computed from unrounded values. † fewer than 50 objects in one dataset: unreliable.

Class	Size	n COCO	n KITTI	YOLO11s	RT-DETR-L	Faster R-CNN
person	small	4308	177	0.360 → 0.024	0.403 → 0.031	0.349 → 0.039
person	medium	3723	530	0.673 → 0.181	0.690 → 0.185	0.592 → 0.202
person	large	2746	214	0.811 → 0.333	0.830 → 0.345	0.675 → 0.337
car	small	1193	1042	0.322 → 0.265	0.384 → 0.332	0.321 → 0.254
car	medium	547	3344	0.639 → 0.405	0.660 → 0.447	0.556 → 0.404
car	large	178	1885	0.739 → 0.565	0.804 → 0.610	0.564 → 0.551
truck	small†	117	34	0.150 → 0.122	0.231 → 0.302	0.189 → 0.547
truck	medium	164	133	0.357 → 0.068	0.483 → 0.183	0.324 → 0.135
truck	large	133	56	0.540 → 0.176	0.626 → 0.257	0.446 → 0.219

5.4 Composition effect. The flat pooled car AP is consistent with a composition effect (hypothesis, not isolated by an experiment): 62% of COCO cars are small versus 17% of KITTI cars, and small objects have much lower AP. KITTI does *not* have more small objects than COCO in this 3-class view: person/car/truck are 43/34/23% small/medium/large on COCO and 17/54/29% on KITTI (measured). Pooled size AP therefore needs care: Faster R-CNN's pooled small-object AP is almost unchanged (0.286 → 0.280, -2.3%), but this is driven by 34 small KITTI trucks (small-truck AP 0.547 versus medium 0.135), which is noise. I therefore do **not** claim that Faster R-CNN is robust on small objects.

5.5 Input-size confound and the 1280 experiment. The three models do not see KITTI at the same effective input (measured on one 1242×375 image, `results/input_shapes.json`). YOLO11s is letterboxed with an aspect-preserving rectangle: 224×640, a scale of 0.52 on both axes (padding included in the size). RT-DETR-L is stretched to 640×640 with no padding, so aspect ratio is not preserved: its scale is 0.52 horizontally but 1.71 vertically.

Faster R-CNN is resized to 402×1333, a scale of 1.07 on both axes. Network input sizes are 143,360, 409,600 and 559,104 pixels for YOLO11s, RT-DETR-L and Faster R-CNN (derived). RT-DETR-L-versus-YOLO11s comparisons are therefore confounded by input size as well as architecture, not only the comparisons with Faster R-CNN. To test input size for one model, I re-ran YOLO11s on KITTI at imgsz 1280 (416×1280, 532,480 pixels): 3-class AP rose from 0.241 to 0.262 (+0.021, +8.7%) and small-object AP from 0.137 to 0.215. Its pixel count is now 1.30× RT-DETR-L's, closer to it than the 640 run's was (122,880 versus 266,240 pixels apart), yet YOLO11s still trails RT-DETR-L by 0.023 AP; it trails Faster R-CNN by only 0.004, within what I consider noise. A hypothesis, not tested here, is that input pixel count alone does not explain RT-DETR-L's advantage over YOLO11s; the other two models were not re-run at other sizes. By class (measured, single run), person AP was unchanged at 1280 (0.196 vs 0.197) while car (+0.021) and truck (+0.041) improved, so resolution does not explain the pedestrian collapse. The 1280 run's mean latency (17.06 ms) came out lower than the 640 run's (18.8 ms), which cannot reflect its real cost, so I treat it as evidence of timing noise on this laptop and do not use it.

6. Analysis 2: speed-accuracy trade-off and architecture

Figure 3 plots AP against latency. *Measured:* YOLO11s is the fastest and smallest (63.3 FPS on COCO, 53.2 on KITTI, 0.126 GB peak). On COCO, RT-DETR-L has 2.2× YOLO11s' latency for +0.061 AP (+13.1%); on KITTI the AP gain is +0.044 (+18.2%) at 1.8× the latency. Faster R-CNN is the slowest (10.6 FPS on COCO, 15.9 on KITTI) and uses the most memory. RT-DETR-L is faster (2.7× on COCO, 1.9× on KITTI) and at least as accurate as Faster R-CNN on both datasets (+0.106 and +0.019 AP), although the KITTI AP gap is small (Section 5.1), so RT-DETR-L is the most balanced choice here and YOLO11s the choice if speed or memory dominate.

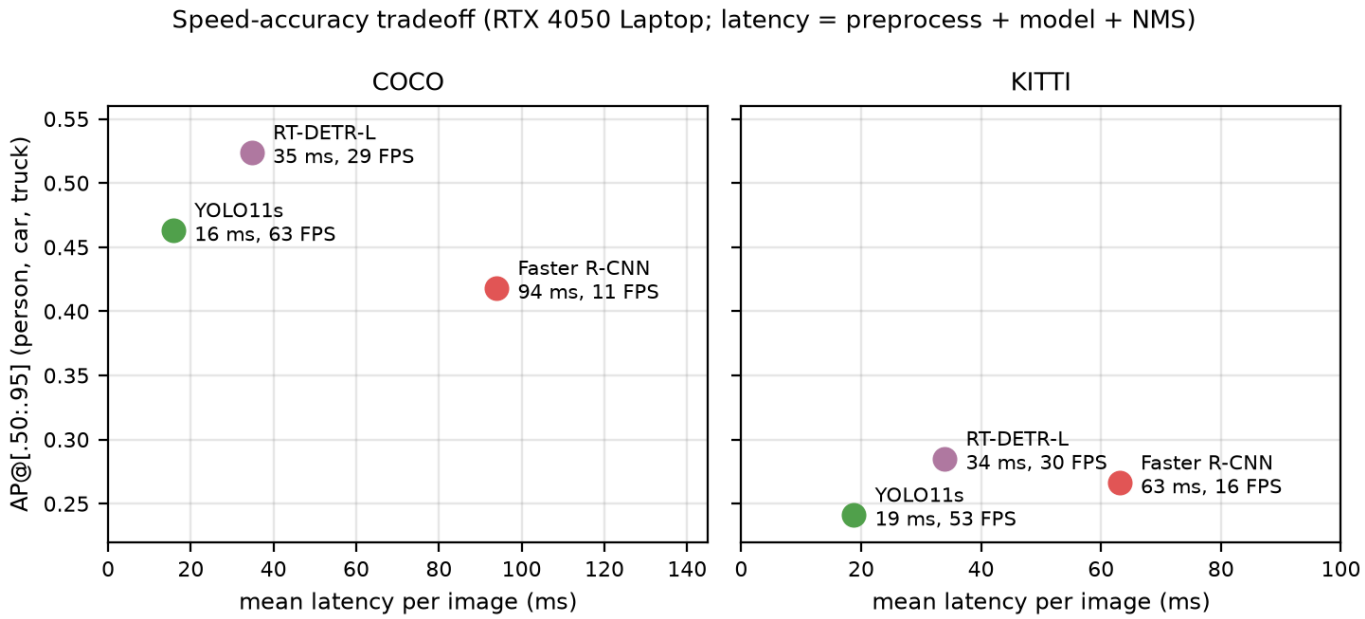


Figure 3. 3-class AP@[.50:.95] versus mean latency per image on COCO (left) and KITTI (right). Latency = preprocessing + model + NMS + copy to CPU, mean after 10 warmup images, on a 45 W laptop GPU; differences under about 20% are treated as noise. The y-axis is truncated at 0.22 to separate the points. No error bars (single run per configuration).

Latency across datasets. From COCO to KITTI the mean latency changes by +19% for YOLO11s, -2% for RT-DETR-L and -33% for Faster R-CNN (Table 6). Only Faster R-CNN's change is larger than my noise threshold; a plausible cause is fewer pixels after torchvision's resize on KITTI's wide, short images (hypothesis, untested). YOLO11s' small increase is unexplained, and RT-DETR-L's near-identical latency and memory on both datasets are consistent with its fixed 640×640 input (hypothesis).

Architecture discussion (hedged). YOLO11s is a single-stage, anchor-free CNN with NMS on the output; RT-DETR-L is query-based with a transformer decoder and needs no NMS [7][8]; Faster R-CNN is two-stage and anchor-based with an RPN and NMS [4][5][6]. The measurements are consistent with the usual expectation that the one-stage model is cheapest and the two-stage model costs the most per image. They do not isolate the cause of RT-DETR-L's better accuracy on both datasets: its checkpoint, parameter count (32.97 M vs 9.46 M for

YOLO11s) and training recipe all differ from YOLO11s, so I cannot attribute the gap to the NMS-free design (hypothesis). Likewise the comparison with Faster R-CNN is confounded by input scale (Section 5.5).

7. Failure analysis

I matched detections to ground truth at score ≥ 0.4 and IoU ≥ 0.5 (`scripts/failure_analysis.py`) and ranked KITTI images by misses common to all models ("missed by all"), by the spread in misses between models ("disagreement"), and by total false positives. A single score cutoff is not the same operating point for every model, because the models' scores are calibrated differently, so the miss and false-positive counts below depend on it. I picked two of the four top images per category that make different points (reasons in `figures/report/SELECTION.md`); they are extreme cases by construction. In Figures 4–9, panels are YOLO11s, RT-DETR-L and Faster R-CNN from top to bottom; blue = correct, red = false positive, orange = missed ground truth, dashed gray = ignore region; class and score labels appear only on large boxes; panels are cropped vertically to the band containing the boxes; and some red boxes may be real but unlabelled objects. Table 8 gives the overall miss rates: YOLO11s misses 55% of small objects versus 22% for Faster R-CNN, and trucks are missed most often by every model.

Table 8. KITTI miss rate (fraction of ground-truth objects with no matching detection) at score ≥ 0.4, IoU ≥ 0.5, by object size and class.

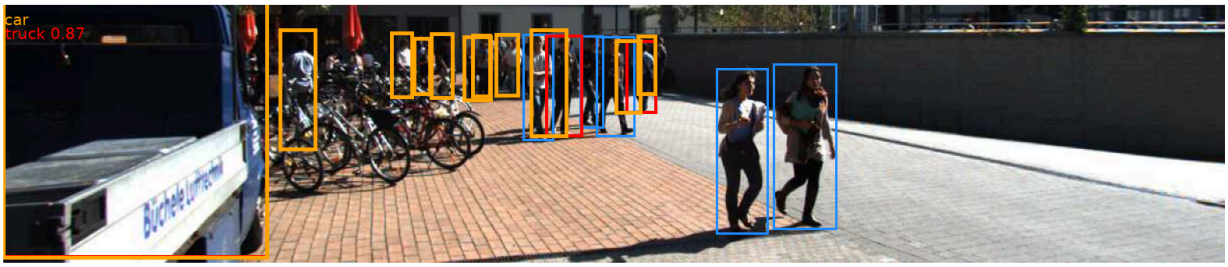
Model	small	medium	large	person	car	truck
YOLO11s	0.554	0.362	0.176	0.489	0.304	0.744
RT-DETR-L	0.297	0.243	0.153	0.403	0.191	0.471
Faster R-CNN	0.223	0.227	0.150	0.347	0.177	0.368

Figure 4 shows a row of distant cars that YOLO11s mostly misses and RT-DETR-L finds. Figure 5 shows a vehicle labelled *car* that all models call *truck* and a crowd of small pedestrians that is mostly missed (consistent with a class-definition mismatch and small, overlapping people; the vehicle's original KITTI class is not visible). Figure 6 shows overlap and truncation among parked cars. In Figure 7, Faster R-CNN finds the most cars but adds many extra boxes; together with Figure 4, this is consistent with its larger input and the miss-rate pattern in Table 8 (hypothesis). In Figures 8 and 9, many red car boxes sit on distant parked cars that appear real but unlabelled (the Ultralytics KITTI export has no DontCare regions), so KITTI AP may understate precision (hypothesis, from visual inspection only); the red person boxes in Figure 9 may be genuine errors, but I cannot tell what those objects are.

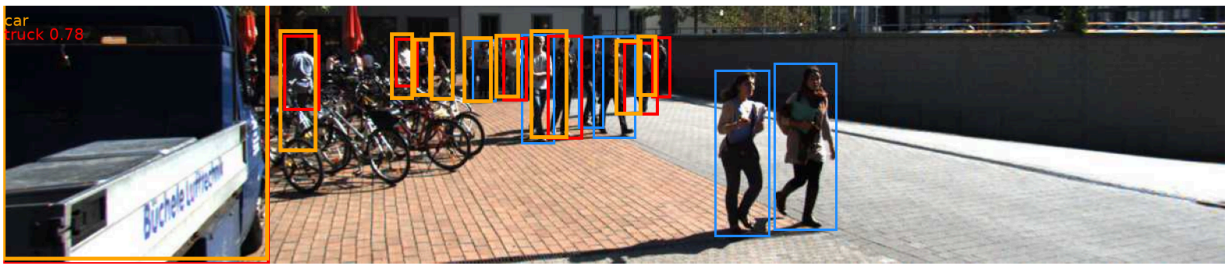


Figure 4. KITTI 003554 (disagreement). Row of distant cars: YOLO11s 1 correct and 13 missed, RT-DETR-L 10 correct, Faster R-CNN 8 correct with 14 false positives.

KITTI 005226: YOLO11s: 5 correct, 3 false positives, 11 missed



RT-DETR-L: 6 correct, 7 false positives, 10 missed



Faster R-CNN: 5 correct, 13 false positives, 11 missed

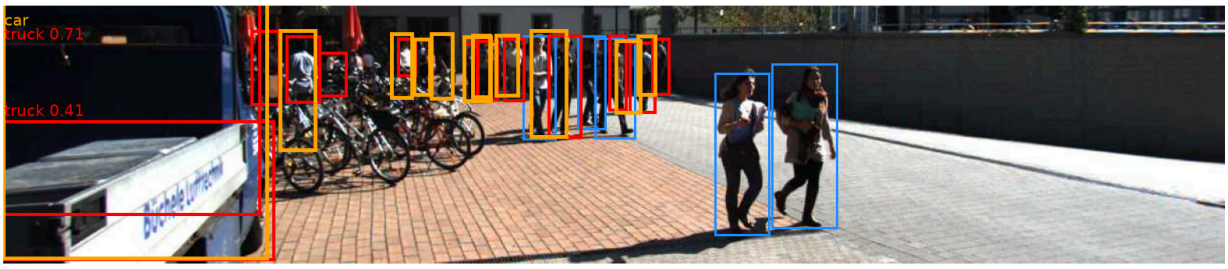
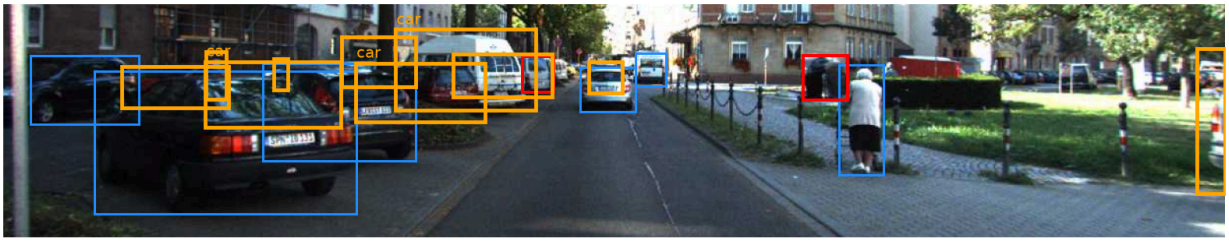
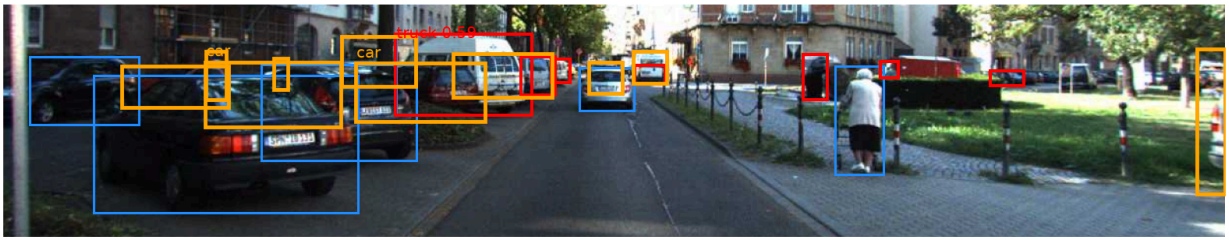


Figure 5. KITTI 005226 (missed by all). A large vehicle labelled car is predicted as truck by all three models; misses are 11/10/11 (YOLO11s/RT-DETR-L/Faster R-CNN).

KITTI 005245: YOLO11s: 6 correct, 2 false positives, 10 missed



RT-DETR-L: 6 correct, 7 false positives, 10 missed



Faster R-CNN: 7 correct, 15 false positives, 9 missed

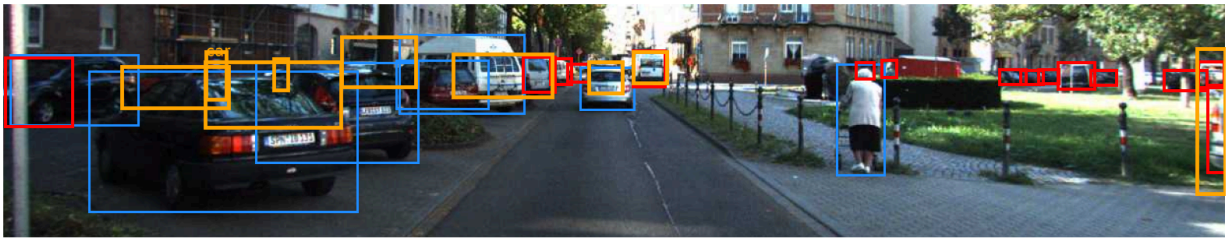
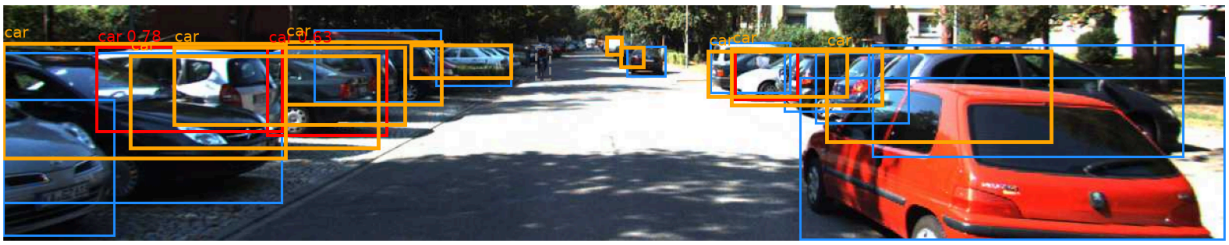
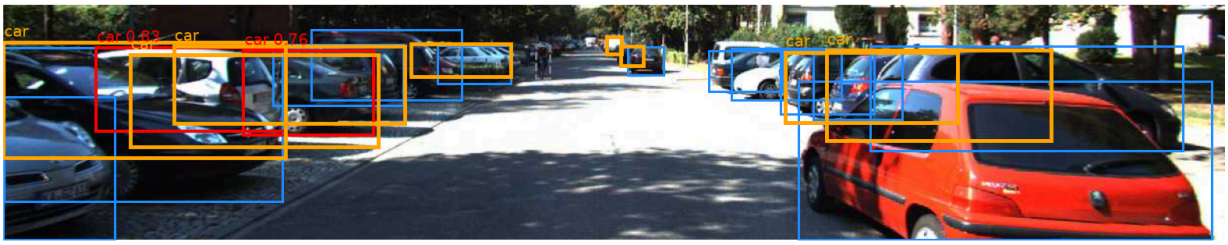


Figure 6. KITTI 005245 (missed by all). Overlapping parked cars, a car truncated at the image border and distant cars are missed; misses are 10/10/9.

KITTI 003219: YOLO11s: 10 correct, 3 false positives, 10 missed



RT-DETR-L: 12 correct, 2 false positives, 8 missed



Faster R-CNN: 16 correct, 19 false positives, 4 missed

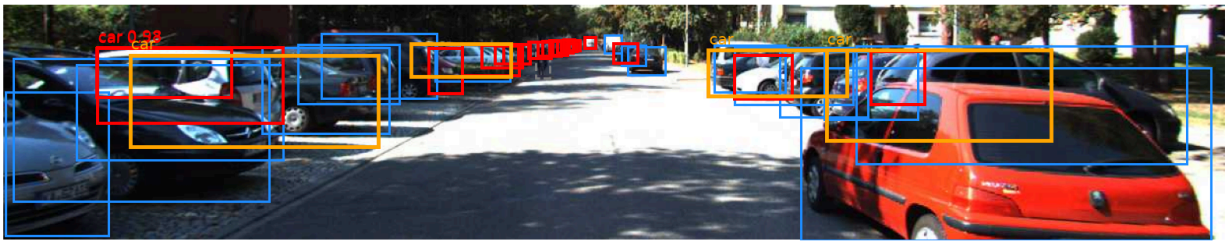


Figure 7. KITTI 003219 (disagreement). Dense parked cars: Faster R-CNN 16 correct, 4 missed, 19 false positives; YOLO11s 10 correct, 10 missed.



Figure 8. KITTI 000018 (false positives). Almost empty road with 2 labelled objects; false positives are 10/11/26.



Figure 9. KITTI 006255 (false positives). Same kind of scene; false positives are 8/10/28, and YOLO11s and Faster R-CNN draw red person boxes on thin upright roadside objects.

8. Limitations and threats to validity

- **Labels.** The Ultralytics KITTI export drops DontCare regions and has no truncation or occlusion flags, so some false positives may be real unlabelled objects. Van is merged into car; cyclist and sitting person are ignore regions; tram and misc are dropped. Only 223 trucks exist in KITTI val.
- **Input size.** The three models see KITTI at three different effective inputs (Section 5.5), so every pairwise comparison is confounded by input size; only YOLO11s was re-run at a higher resolution. The `input_size` field in the RT-DETR-L result files says "letterboxed", which is inaccurate; the measured preprocessing is in `results/input_shapes.json`.
- **Statistics.** One run per configuration, no confidence intervals or bootstrap, so small AP differences between models are not established.
- **Size buckets.** COCO area is mask area, KITTI area is box area, so the buckets are not identical across datasets.
- **Latency.** Measured on a power-limited laptop GPU, excluding image decoding; differences under about 20% are noise. The 1280 run's latency is not used.
- **Failure figures.** Selected extreme cases, judged visually; causes are hypotheses.
- **AR@1** differs between the 80-class and 3-class tables because maxDets is capped per image per class; AR@100 is the meaningful comparison.

9. Conclusions

- All three COCO-pretrained detectors lose a large share of their 3-class AP on KITTI (48%, 46%, 36% for YOLO11s, RT-DETR-L, Faster R-CNN), and the ranking changes (Section 5.1).
- The loss is concentrated in person and truck; the flat pooled car AP hides lower AP at matched sizes and is consistent with a composition effect (Sections 5.2–5.4).
- Faster R-CNN's smaller relative drop and its pooled small-object AP should not be read as robustness: it is partly an input-size confound and small-truck noise. Raising YOLO11s to 1280 recovers part of its deficit (+8.7%) but leaves person AP unchanged, so resolution alone does not explain the pedestrian collapse (single run).
- RT-DETR-L gives the best accuracy-speed balance here; YOLO11s is the cheapest. The KITTI gap between RT-DETR-L and Faster R-CNN is within what I cannot resolve without confidence intervals.
- Many KITTI “false positives” appear to be real unlabelled objects, so KITTI AP is a conservative estimate (hypothesis). Fine-tuning on KITTI and per-class statistics with confidence intervals are the obvious next steps.

10. References

1. T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollar, C. L. Zitnick, "Microsoft COCO: Common Objects in Context," ECCV 2014, pp. 740-755 (Springer). https://arxiv.org/abs/1405.0312
2. A. Geiger, P. Lenz, R. Urtasun, "Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite," CVPR 2012, pp. 3354-3361. https://www.cvlibs.net/publications/Geiger2012CVPR.pdf
3. A. Geiger, P. Lenz, C. Stiller, R. Urtasun, "Vision meets Robotics: The KITTI Dataset," International Journal of Robotics Research (IJRR), vol. 32, no. 11, 2013. https://www.cvlibs.net/publications/Geiger2013IJRR.pdf
4. S. Ren, K. He, R. Girshick, J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," NeurIPS 2015 (Advances in Neural Information Processing Systems 28), pp. 91-99. https://arxiv.org/abs/1506.01497
5. T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, S. Belongie, "Feature Pyramid Networks for Object Detection," CVPR 2017, pp. 2117-2125. https://arxiv.org/abs/1612.03144
6. K. He, X. Zhang, S. Ren, J. Sun, "Deep Residual Learning for Image Recognition," CVPR 2016, pp. 770-778. https://arxiv.org/abs/1512.03385
7. N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, S. Zagoruyko, "End-to-End Object Detection with Transformers," ECCV 2020. https://arxiv.org/abs/2005.12872
8. Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, J. Chen, "DETRs Beat YOLOs on Real-time Object Detection," CVPR 2024, pp. 16965-16974. https://arxiv.org/abs/2304.08069
9. J. Redmon, S. Divvala, R. Girshick, A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," CVPR 2016, pp. 779-788. https://arxiv.org/abs/1506.02640
10. Ultralytics, "YOLO11" model documentation. YOLO11 has no paper, so this page is the citation. https://docs.ultralytics.com/models/yolo11/
11. Ultralytics, "RT-DETR" model documentation. https://docs.ultralytics.com/models/rtdetr/
12. Ultralytics repository. https://github.com/ultralytics/ultralytics
13. PyTorch, torchvision detection models documentation, fasterrcnn_resnet50_fpn. https://docs.pytorch.org/vision/stable/models/faster_rcnn.html
14. COCO API repository (pycocotools source). https://github.com/cocodataset/cocoapi
15. KITTI Vision Benchmark Suite, 2D object detection benchmark. https://www.cvlibs.net/datasets/kitti/eval_object.php?obj_benchmark=2d

AI assistants (Claude) helped write the code and draft this report, and ran some analysis scripts on my machine at my direction; I ran the main evaluations and reviewed the results.